

A REPORT ON

**DESIGN AND FPGA IMPLEMENTATION OF HYBRID
VEDIC WALLACE TREE MULTIPLIER**



Prepared By

Rachit Kumar (2348026)
Ayush Kumar (2304134)
Roshani Kumari (2304145)

Supervised By

Dr. Gayadhar Pradhan
Associate Professor, ECE Department

Dr. Bal Chand Nagar
Assistant Professor, ECE Department

Acknowledgement

We would like to express our sincere gratitude to all the instructors, faculty members, and colleagues who contributed to the successful completion of this minor project.

We are deeply thankful to our supervisor, Dr. Gayadhar Pradhan, for his invaluable mentorship, continuous encouragement, and guidance throughout the course of this project. His insights and support played a crucial role in shaping our understanding and approach.

We would also like to extend our sincere thanks to Dr. Bal Chand Nagar for his technical guidance and support, which greatly assisted us in overcoming various challenges during the implementation phase.

Furthermore, we would like to acknowledge and express our appreciation to the authors of the research journal that served as a primary reference for this work. Their detailed analysis, innovative approach, and comprehensive study on hybrid Vedic–Wallace multipliers and their FPGA implementation provided a strong foundation and significant inspiration for our project.

We are also grateful to our institution for providing the necessary resources, tools, and environment to carry out this work successfully.

Finally, we thank our peers and friends for their support, collaboration, and constructive discussions throughout the project.

Abstract

Multiply–Accumulate (MAC) operations form the computational backbone of digital signal processing (DSP) systems and machine learning accelerators, where efficient multiplication plays a critical role in determining overall performance. In such systems, low latency, reduced power consumption, and minimal hardware area are key design requirements.

In this work, a hybrid multiplier architecture is proposed by combining the advantages of the Vedic multiplication algorithm and the Wallace tree reduction technique. The proposed design exploits the parallelism of Vedic multiplication for partial product generation and the efficiency of Wallace tree structures for fast partial product reduction.

The proposed multiplier is implemented on an Artix-7 FPGA platform for multiple word lengths, and its performance is evaluated in terms of hardware utilization and power consumption. Experimental results show that, for an 8-bit implementation, the proposed hybrid design reduces LUT utilization by approximately 13–20% and dynamic power consumption by about 6–10% compared to conventional Vedic and Wallace multipliers. Similar improvements are observed for higher word lengths, demonstrating better scalability.

Further optimization is achieved through the elimination of redundant computations and the application of clock and power gating techniques, leading to additional reductions in dynamic power consumption.

Overall, the proposed architecture achieves a balanced trade-off between speed, area, and power, making it well suited for high-performance and energy-efficient DSP applications.

Keywords- vedic multiplier, wallace tree multiplier, hybrid multiplier, partial product, ripple carry adder, dynamic power, FPGA, VLSI

Contents

1	Introduction	6
2	Motivation	7
2.1	Array Multiplier	7
2.2	Vedic Multiplier	8
2.3	Wallace Tree Multiplier	9
3	Literature Survey	11
4	Objective	12
5	Proposed 8-bit Hybrid Vedic-Wallace Tree Multiplier	13
5.1	Proposed HVWTM Architecture	14
5.2	Example	16
6	Simulation Results and Discussions	18
6.1	Schematic of Proposed HVWTM	18
6.2	Verilog Implementation	20
6.3	Simulation Waveform Analysis	21
7	Inferences	23
7.1	Utilization	23
7.2	Timing	24
7.3	Power	25
8	Conclusion	26
	References	27
	Appendix A	28
	Appendix B	32

List of Figures

2.1	4x4 Array Multiplier	8
2.2	Urdhva Tiryakbhyam for a 3x3 decimal multiplication	8
2.3	4x4 Vedic Multiplier	9
2.4	4x4 Wallace Tree Multiplier	10
5.1	Proposed partial product reduction strategy for 8x8 multiplication	14
5.2	Proposed Architecture of NxN HVWTM Multiplier	15
5.3	Architecture of Proposed 8x8 HVWTM Multiplier	16
5.4	Illustration of scheme of multiplication by proposed HVWTM architecture . . .	17
6.1	8x8 Hybrid Vedic Wallace Tree Multiplier Schematic	19
6.2	RTL Schematic	21
6.3	Functional Verification of Proposed HVWTM	22

List of Tables

7.1	Device utilization of HVWTM	24
7.2	Timing Summary of HVWTM at 100 MHz clock frequency	24
7.3	Power consumption of HVWTM for Artix-7 family board (xc7a35ticpg236-1L)	25

Chapter 1

Introduction

A multiplier is a fundamental combinational circuit in digital electronics that is used to perform multiplication of binary numbers. It plays a crucial role in applications such as digital signal processing (DSP), microprocessors, image processing, and communication systems.

In binary multiplication, the process is similar to decimal multiplication, where partial products are generated and then summed to obtain the final result. Each bit of one operand is multiplied with every bit of the other operand, and the results are appropriately shifted and added.

Multipliers are widely used in systems where high-speed arithmetic operations are required. The efficiency of a multiplier significantly affects the overall performance of a digital system, especially in real-time applications.

In modern digital systems, multipliers play a critical role in determining overall system performance, particularly in applications such as digital signal processing, image processing, and high-speed communication systems. These applications demand arithmetic units that are not only fast but also area-efficient and power-efficient.

Conventional multiplier architectures, such as the Array Multiplier, Vedic Multiplier, and Wallace Tree Multiplier, offer improvements in specific performance parameters. However, each of these architectures involves certain trade-offs. For instance, Array Multipliers are simple but suffer from high delay, Vedic Multipliers improve speed but may not be optimal in terms of area, and Wallace Tree Multipliers provide high speed at the cost of increased hardware complexity.

Chapter 2

Motivation

Multipliers are designed with different architectures to optimize parameters such as speed, area, and power consumption. The choice of multiplier depends on the specific requirements of the digital system. The commonly used types of multipliers include Array Multiplier, Vedic Multiplier, and Wallace Tree Multiplier.

- **Array Multiplier:** Uses a regular structure where partial products are generated and added sequentially. It is simple but relatively slower.
- **Vedic Multiplier:** Based on the *Urdhva Tiryakbhyam* method, it enables faster computation through parallel processing.
- **Wallace Tree Multiplier:** Reduces partial products efficiently using a tree structure, resulting in high-speed performance.

2.1 Array Multiplier

The Array Multiplier is a basic and widely used multiplier architecture in digital systems. It generates partial products using AND gates and sums them using a structured arrangement of adders.

Its regular layout makes it easy to design and suitable for hardware implementation. However, due to the sequential addition of partial products, it has higher propagation delay compared to advanced multipliers.

Despite its slower speed, it is preferred in applications where simplicity and reliability are more important than performance.

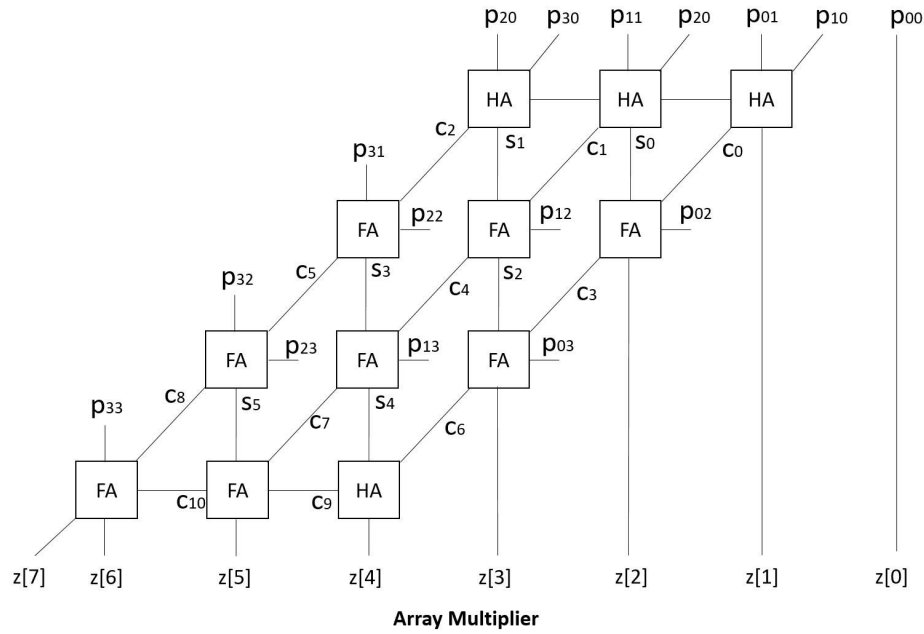


Figure 2.1: 4x4 Array Multiplier

2.2 Vedic Multiplier

The Vedic Multiplier is based on the Urdhva Tiryakbhyam (Vertically and Crosswise) algorithm from Vedic mathematics. It allows simultaneous generation of partial products, improving computational speed.

Due to its parallel nature, the Vedic Multiplier reduces delay and enhances efficiency. It is particularly useful in high-speed digital applications and modern processor design.

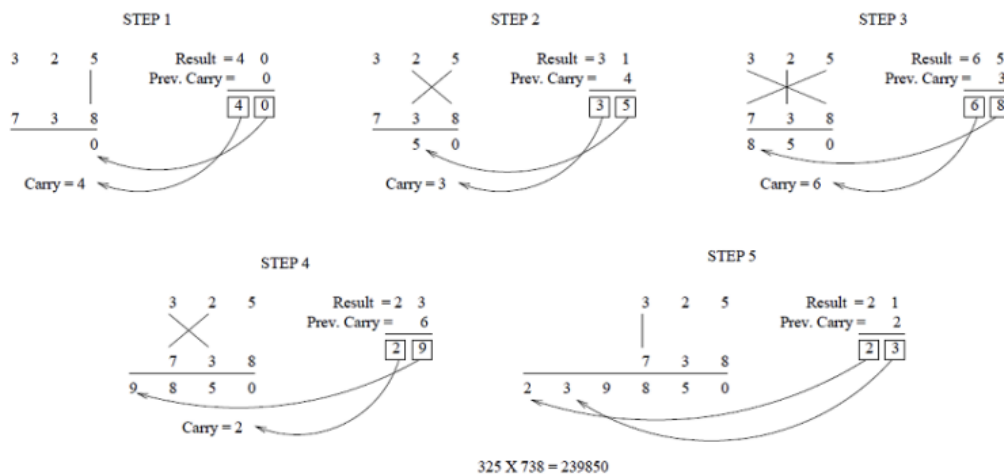


Figure 2.2: Urdhva Tiryakbhyam for a 3x3 decimal multiplication

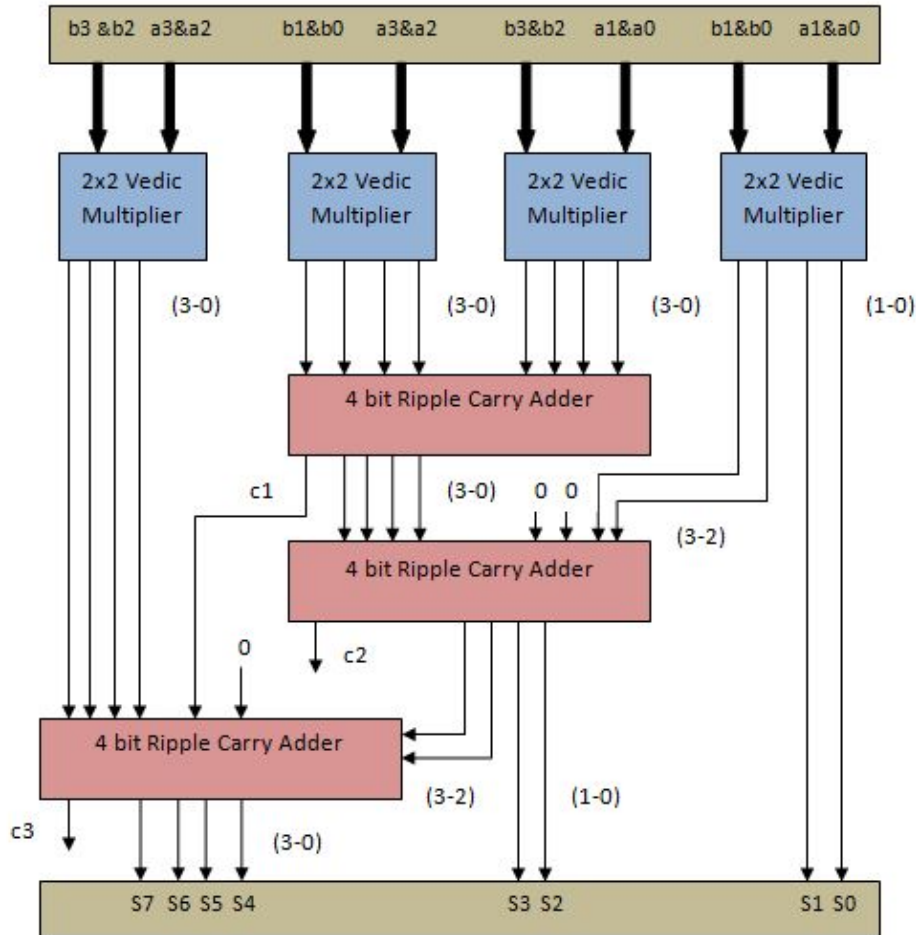


Figure 2.3: 4x4 Vedic Multiplier

2.3 Wallace Tree Multiplier

The Wallace Tree Multiplier is a high-speed multiplication technique that reduces the number of sequential addition stages by organizing partial products in a tree structure.

Instead of adding partial products one by one, it combines them in parallel using half adders and full adders, significantly reducing computation time.

Although it offers high performance, the design is more complex and requires greater hardware resources compared to simpler multiplier architectures.

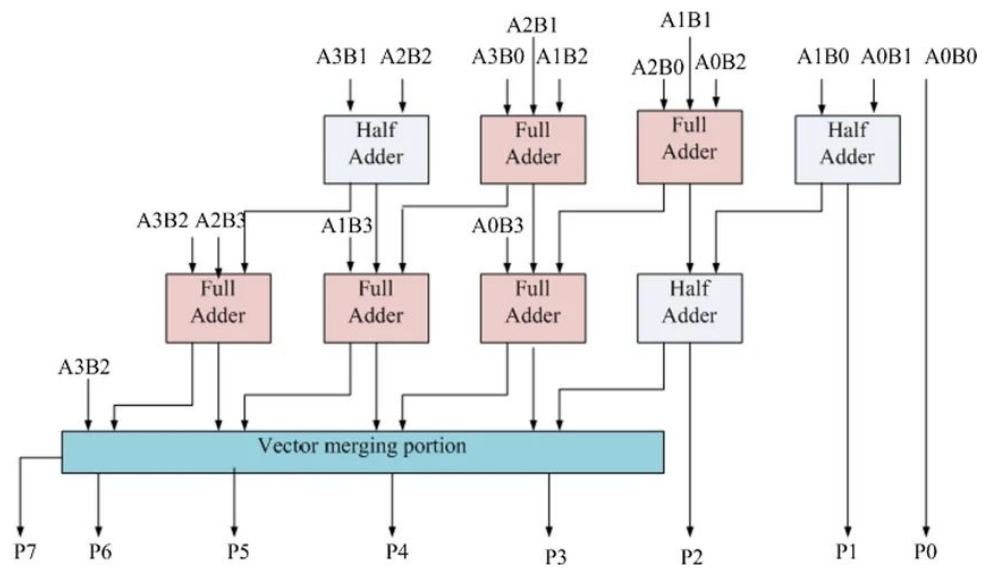


Figure 2.4: 4x4 Wallace Tree Multiplier

Chapter 3

Literature Survey

In recent years, Vedic multipliers based on the Urdhva Tiryagbhyam algorithm have gained popularity due to their modular structure and inherent parallelism. These multipliers offer reduced power consumption and improved energy efficiency. However, they require a larger number of hardware resources compared to other multiplier architectures.²

On the other hand, Wallace tree multipliers are widely recognized for their high-speed operation and reduced hardware utilization. By efficiently reducing partial products using a tree structure of full adders and half adders, Wallace multipliers achieve lower area and faster computation. However, due to longer interconnects, they tend to exhibit higher power consumption, especially for larger word lengths.³

To address these trade-offs, hybrid multiplier architectures have been proposed that combine the advantages of both Vedic and Wallace multipliers. The hybrid Vedic–Wallace tree multiplier (HVWTM) leverages the modularity and low-power characteristics of Vedic multiplication at higher levels, while employing Wallace tree structures for efficient partial product reduction at lower levels. This approach results in a balanced design that achieves both reduced area and lower power consumption.¹

Additionally, recent works have introduced further optimizations such as power gating and zero-detection techniques to minimize switching activity in multiplier circuits. These techniques significantly reduce dynamic power consumption, making the design suitable for Machine Learning Accelerators.^{2,4}

Overall, the literature indicates that hybrid multiplier architectures provide an effective solution for achieving high performance, low power consumption, and reduced hardware complexity suitable for Hardware Accelerators.

Chapter 4

Objective

The main objective of this design is to combine the advantages of both Vedic and Wallace Tree architectures while minimizing their individual drawbacks.

The proposed hybrid approach focuses on reducing propagation delay by utilizing parallel partial product generation from the Vedic multiplication technique, while efficiently compressing these partial products using the Wallace Tree structure. This results in faster computation compared to traditional multipliers.

In addition to speed improvement, the design also aims to optimize hardware utilization, thereby reducing the overall area of the chip. Furthermore, by minimizing unnecessary switching activities and optimizing logic operations, the proposed multiplier contributes to lower power consumption.

Hence, the Hybrid Vedic Wallace Tree Multiplier provides a balanced solution that achieves reduced delay, optimized area, and lower power consumption, making it highly suitable for high-performance and energy-efficient digital systems.

Chapter 5

Proposed 8-bit Hybrid Vedic-Wallace Tree Multiplier

The partial product reduction strategy proposed by Wallace³ focused on increasing the multiplication speed of the multiplier. The improvements to the Wallace scheme in successive WTM architectures have majorly targeted the critical path optimization. Hence, WTM is widely considered to be speed optimized multiplier design.⁴ However, it is well understood from the architecture of WTM that as the wordlength increases, the number of partial products and the size of Wallace Tree also increase, resulting in longer interconnects.⁵ It is estimated that 50% of dynamic power consumption is due to interconnects. On the other hand, VM architecture because of its modularity enjoys the advantage of shorter interconnects.^{9,10}

The architecture of VM is such that even when wordlength increases, the length of interconnects is not increased proportionately.⁷ WTM is an area-optimized architecture with its implementation consuming the minimum number of HAs and FAs compared to other digital multiplier architectures. VM architecture, despite being powerefficient, consumes more hardware resources. This can be attributed to more number of HAs required for implementation, which essentially do not reduce the number of partial product bits.

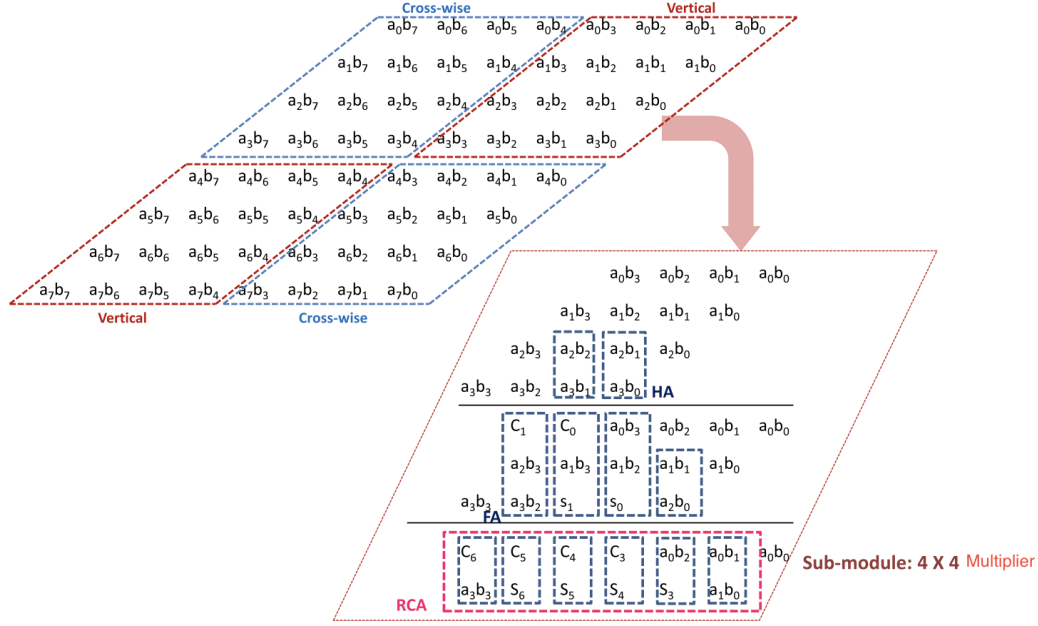


Figure 5.1: Proposed partial product reduction strategy for 8x8 multiplication

Moreover, being a modular architecture designed in a bottom-up approach, VM ends up in a large number of HAs due to the compounding effect. These are critical considerations in the design of the proposed HVWTM architecture. The proposed HVWTM architecture uses WTM for the implementation of lower wordlength blocks. This benefits in minimization of HAs and FAs, and ultimately, the hardware resources required for implementation are optimized.

5.1 Proposed HVWTM Architecture

The proposed Hybrid Vedic Wallace Tree Multiplier (HVWTM) architecture is designed to overcome the individual limitations of conventional multiplier designs by intelligently combining Vedic multiplication and Wallace Tree reduction techniques.

A key observation in this architecture is the division of input operands into smaller segments, allowing multiple smaller multipliers to operate in parallel. This parallelism, derived from the Vedic multiplication approach, significantly reduces computation time compared to sequential multiplication methods.

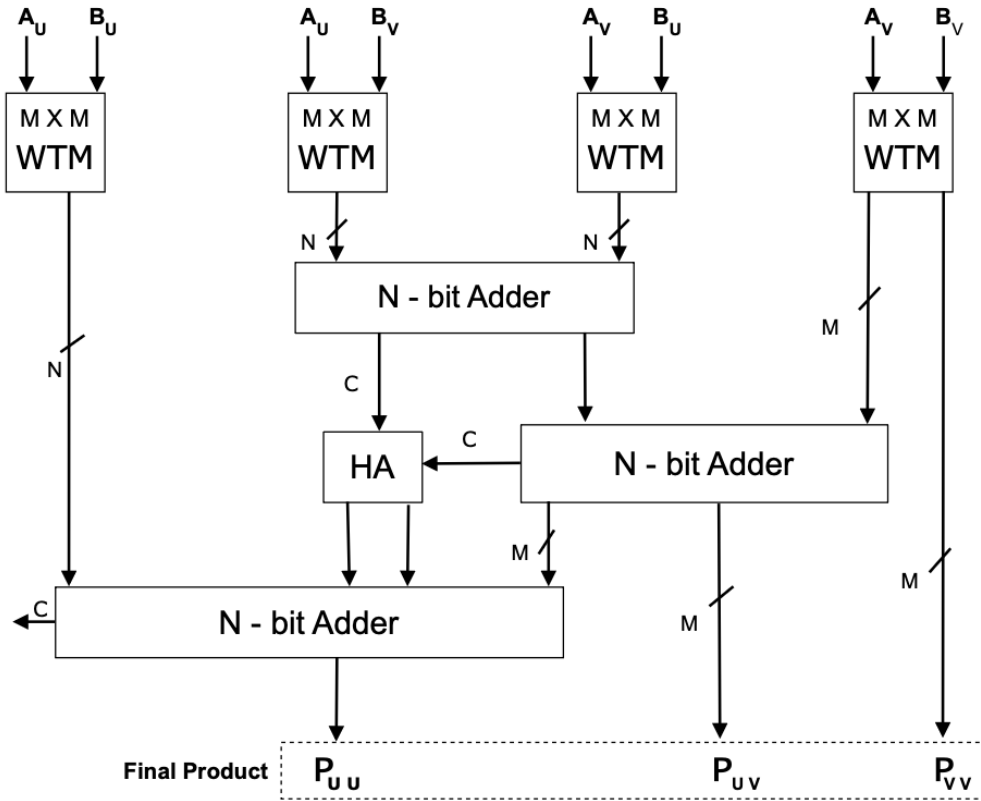


Figure 5.2: Proposed Architecture of $N \times N$ HVWTM Multiplier

Another important aspect of the design is the efficient handling of partial products. Instead of performing sequential addition, the architecture utilizes Wallace Tree-based reduction, where partial products are compressed simultaneously. This reduces the number of addition stages, directly minimizing propagation delay.

It is also noteworthy that the architecture avoids unnecessary hardware complexity by balancing the number of adders used in the reduction stage. The use of N -bit adders and half adders is carefully structured to ensure that performance is improved without excessive increase in area.

Furthermore, the design inherently reduces switching activity due to its optimized data flow, which contributes to lower power consumption. This makes the architecture suitable for applications where energy efficiency is as important as speed.

Thus, the proposed HVWTM architecture stands out by achieving a practical balance between speed, area, and power, rather than optimizing only a single parameter at the cost of others.

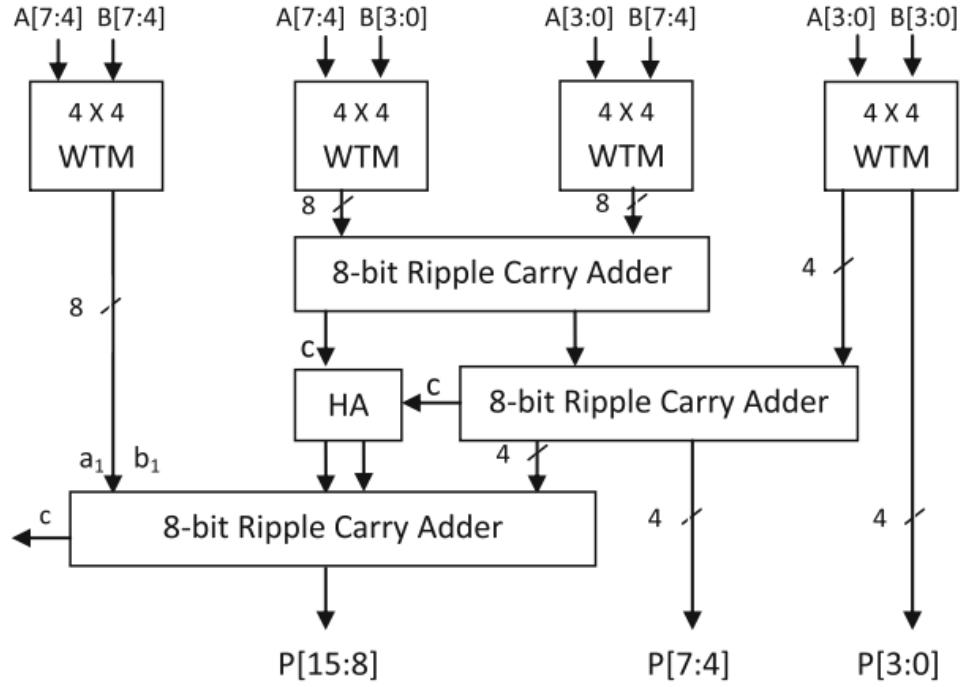


Figure 5.3: Architecture of Proposed 8x8 HVWTM Multiplier

5.2 Example

Let $A = 11011011$ and $B = 10110010$ be the two 8-bit numbers to be multiplied. The scheme of multiplication of these numbers with the proposed HVWTM architecture is explained in Figure 3.4. In the proposed scheme, the both multiplicand A and multiplier B are disintegrated into two sets of 4 bits ($A_U = 1101$, $A_V = 1011$, $B_U = 1011$, and $B_V = 0010$). The top level multiplication is carried out by Urdhva Triyagbhyam Sutra (Vedic multiplication algorithm), while lower hierarchy multiplications are executed using the Wallace scheme.

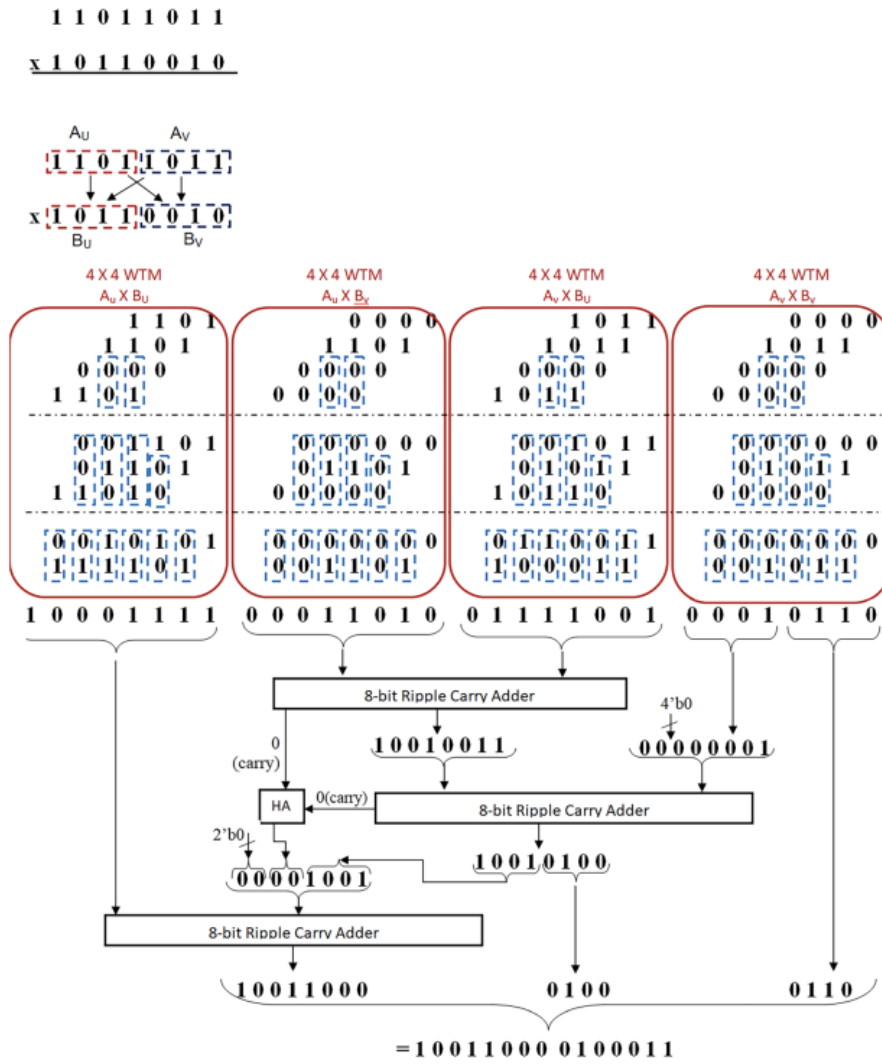


Figure 5.4: Illustration of scheme of multiplication by proposed HVWTM architecture

Chapter 6

Simulation Results and Discussions

The proposed architecture is implemented on the Artix-7 FPGA xc7a35ticpg236-1L platform for rapid prototyping using Xilinx tools. The selected FPGA, fabricated using 28 nm high-K metal gate technology, comprises approximately 20,800 LUTs, 41,600 FFs, and 90 DSPs, and operates over a supply voltage range of 1.0 to 3.3 V. Synthesis and implementation of the design are carried out using the Vivado Design Suite 2025.1. Functional verification along with timing analysis is performed at multiple stages, including Register Transfer Level (RTL), post-synthesis, and post-implementation, to ensure correctness and performance of the design.

6.1 Schematic of Proposed HVWTM

Figure 5.1 illustrates the detailed schematic of the proposed Hybrid Vedic Wallace Tree Multiplier (HVWTM). The schematic represents the complete gate-level implementation of the multiplier, including partial product generation, reduction stages, and final addition.

At the initial stage, input bits are processed through multiple AND gates to generate partial products. These partial products are formed in parallel, reflecting the Vedic multiplication approach, which significantly improves computational speed.

In Figure 5.2, a notable observation in the schematic is the dense network of interconnections in the intermediate stages. This structure corresponds to the Wallace Tree reduction mechanism, where partial products are compressed simultaneously using combinations of half adders and full adders. This parallel reduction effectively reduces the number of sequential addition levels, thereby minimizing propagation delay.

The later stages of the schematic consist of structured N-bit adders that combine the reduced partial products to produce the final output. These adders ensure accurate and efficient accu-

mulation of results.

Although the schematic appears complex due to the large number of interconnected components, this complexity is a direct consequence of optimized parallel processing and efficient hardware utilization. The design avoids unnecessary redundancy while maintaining high performance.

Overall, the schematic clearly demonstrates the integration of Vedic parallel multiplication with Wallace Tree reduction, resulting in a high-speed, area-efficient, and power-optimized multiplier architecture.

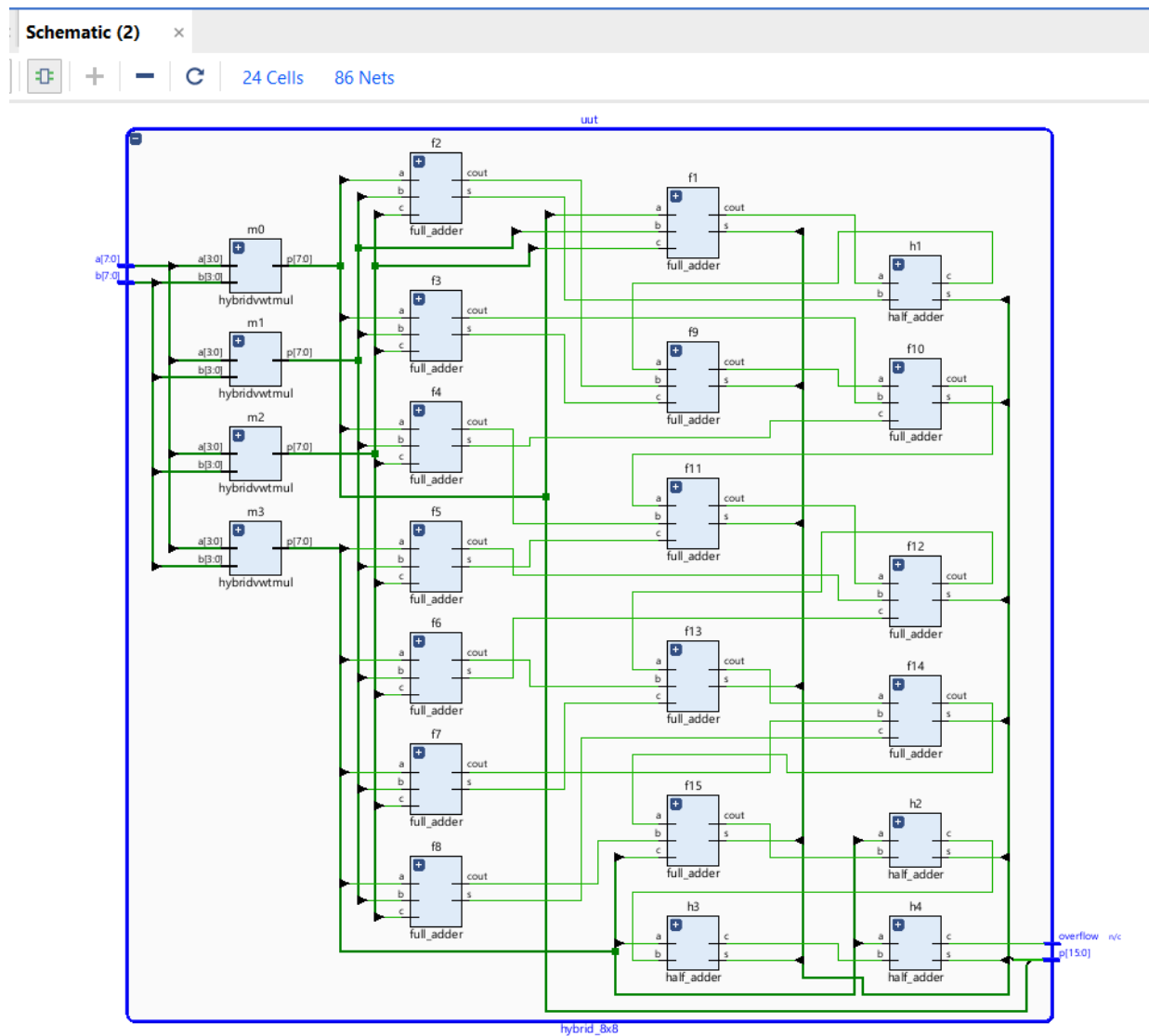


Figure 6.1: 8x8 Hybrid Vedic Wallace Tree Multiplier Schematic

6.2 Verilog Implementation

The proposed HVWTM is implemented using Verilog Hardware Description Language. The design module takes two 8-bit inputs *a* and *b*, and produces an 16-bit output *p*, representing the multiplication result.

The design follows a structured approach where partial products are generated using logical AND operations and then reduced using a combination of half adders and full adders. This reflects the integration of Vedic multiplication and Wallace Tree reduction techniques.

The half adder module computes sum and carry using XOR and AND operations, respectively. The full adder module extends this functionality by including an additional carry input, enabling efficient multi-level addition during partial product reduction.

To verify the functionality of the design, a testbench is developed. The testbench systematically applies all possible input combinations using nested loops, ensuring exhaustive testing of the multiplier.

The simulation uses the `$monitor` statement to display input and output values in real time, while `$dumpfile` and `$dumpvars` are used to generate waveform files for analysis.

A notable aspect of the testbench is the use of iterative loops, which automatically test multiple input combinations without manual intervention. This ensures thorough validation and reduces the chances of missing edge cases.

Overall, the Verilog implementation and simulation confirm that the proposed HVWTM design performs accurate multiplication with efficient hardware utilization.

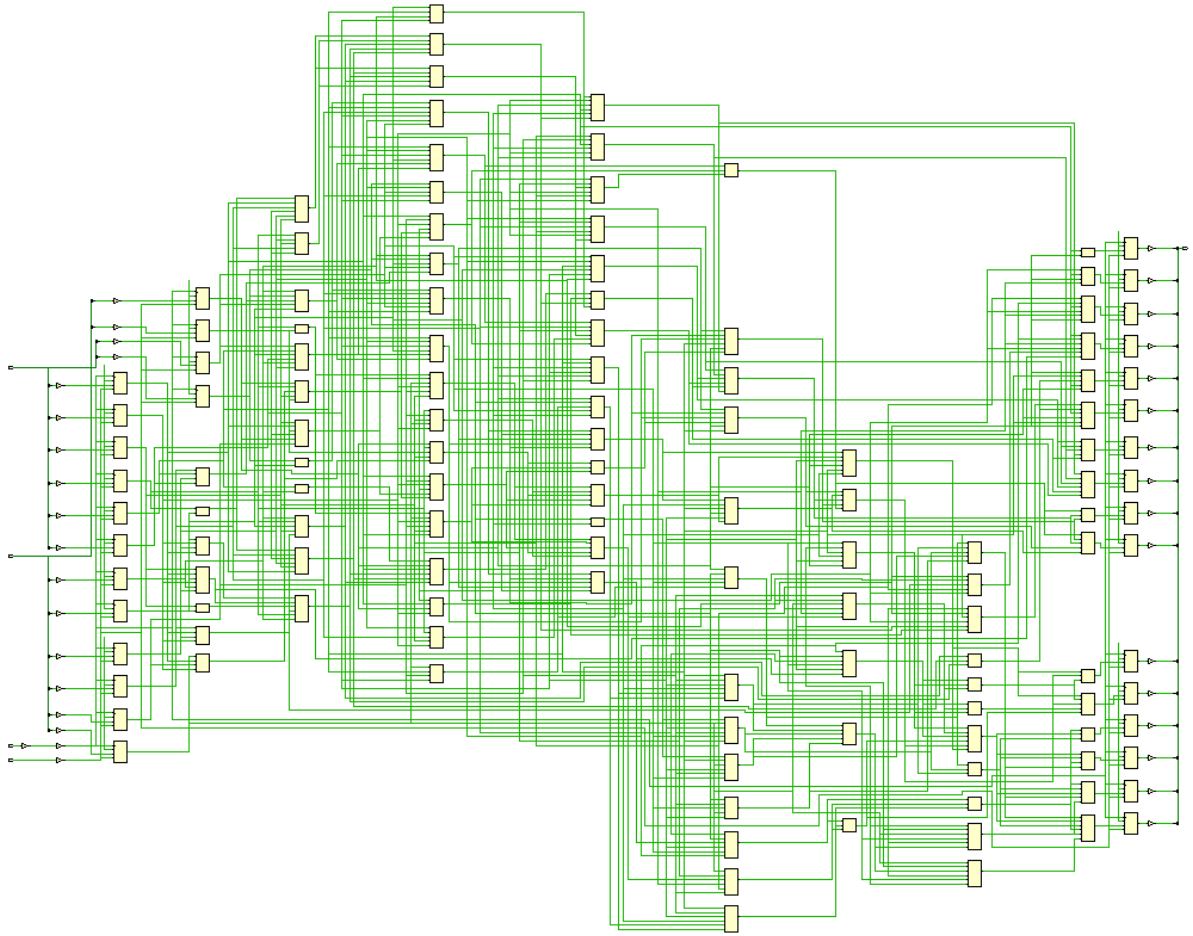


Figure 6.2: RTL Schematic

6.3 Simulation Waveform Analysis

The simulation waveform of the proposed HVWTM is shown in Figure 5.3. The waveform illustrates the behavior of input signals a and b , along with the corresponding output p over time.

A clock signal is applied to synchronize the operation, while the reset signal ensures proper initialization of the system. The input operands a and b are varied systematically during simulation to verify the correctness of the multiplier across multiple test cases.

A key observation from the waveform is that for every change in the input values, the output p reflects the correct multiplication result after a small propagation delay. This confirms the functional correctness of the proposed HVWTM design.

It is also noteworthy that the output transitions are stable and do not exhibit unwanted glitches, indicating proper synchronization and efficient reduction of intermediate signals within the architecture.

The waveform demonstrates that the multiplier operates correctly for different combinations of inputs, thereby validating the reliability and accuracy of the proposed design.

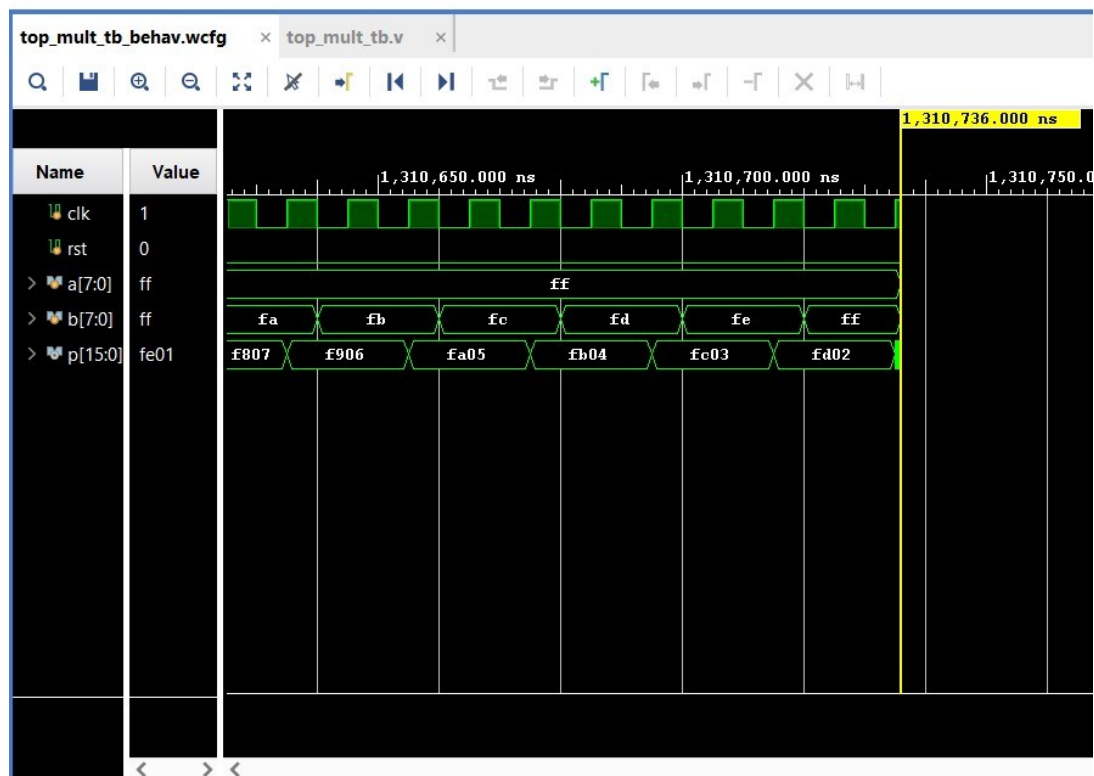


Figure 6.3: Functional Verification of Proposed HVWTM

Chapter 7

Inferences

The schematics and the RTL results for each of the blocks were given in the previous chapter. However, just the results alone are insufficient to gain a complete understanding of the project. Rather, knowing what blocks/what codes/what logic caused these results and how they caused the results to come, can help us to evaluate it as well as predict the outcomes for optimization if the project is expanded to a higher scale in the future.

7.1 Utilization

Resource utilization is an important metric in evaluating hardware efficiency. It indicates how much of the available FPGA resources are consumed by the design.

From the implementation results, the proposed HVWTM utilizes a limited number of hardware resources, demonstrating efficient design optimization. The utilization summary shows that 79 Look-Up Tables (LUTs), 32 Flip-Flops (FFs), and 34 I/O pins are used.

The percentage utilization is very low for LUTs (0.38%) and Flip-Flops (0.08%), indicating that the design is highly area-efficient and leaves significant room for scalability. Although I/O utilization is comparatively higher (32.08%), it is acceptable as it depends on input-output interfacing requirements.

Table 7.1: Device utilization of HVWTM

Wordlength	LUTs	(% Util)	FFs	(% Util)	IOBs
	(N bits)	(logic)			
4	15	0.07	16	0.04	18
8	79	0.38	32	0.08	34

This efficient utilization highlights that the proposed HVWTM achieves optimized hardware usage without unnecessary resource consumption.

7.2 Timing

Timing analysis is crucial to determine the speed and reliability of the design. It evaluates parameters such as setup slack, hold slack, and propagation delay.

In Table 6.2, the maximum operable frequency for given multiplier under the slack constraints has been calculated as $F_{max} = 1 / (\text{ActualClockPeriod} - \text{WNS})$. From the timing report, the Worst Negative Slack (WNS) is observed to be 2.637 ns, and the Worst Hold Slack (WHS) is 0.293 ns. Since both values are positive, it indicates that the design meets all timing constraints successfully. We have taken clock period as 10 ns, so F_{max} is 100 MHz.

Additionally, the Total Negative Slack (TNS) is zero, and no failing endpoints are reported. This confirms that the proposed HVWTM operates without timing violations.

The detailed path analysis further shows that the logic delay (26% of total delay) and route delay(74% of total delay) are well balanced, contributing to stable and efficient operation. Overall, the timing performance demonstrates that the design achieves reduced delay and reliable high-speed operation.

Table 7.2: Timing Summary of HVWTM at 100 MHz clock frequency

Wordlength	WNS	WHS	Total Delay	Logic Delay	Route Delay
(N bits)	(in ns)	(in ns)	(in ns)	(in ns)	(in ns)
4	5.456	0.265	4.515	1.182	3.333
8	2.637	0.293	7.335	1.930	5.405

7.3 Power

Power consumption is a critical parameter, especially for portable and energy-efficient systems. The power analysis of the proposed HVWTM provides insights into its energy efficiency.

The total on-chip power consumption is observed to be 0.063 W. Out of this, the major portion (91%) is contributed by static power, while dynamic power accounts for only 9%.

Among dynamic components, clock power contributes approximately 12%, while logic and signal power remain minimal. This indicates that the design effectively minimizes switching activity, leading to reduced dynamic power consumption.

Table 7.3: Power consumption of HVWTM for Artix-7 family board (xc7a35ticpg236-1L)

Wordlength (N bits)	Static power (W)	Signal power (W)	Logic power (W)	IO power (W)	Dynamic power (W)	Total power (W)
4	0.062	0.001	0.001	0.006	0.006	0.068
8	0.060	0.001	0.001	0.002	0.003	0.063

The low overall power consumption confirms that the proposed HVWTM is suitable for low-power applications while maintaining high performance.

Chapter 8

Conclusion

In this project, a Hybrid Vedic Wallace Tree Multiplier (HVWTM) has been successfully designed, implemented, and analyzed to achieve improved performance over conventional multiplier architectures. The proposed design effectively combines the advantages of Vedic multiplication, which enables parallel partial product generation, and Wallace Tree reduction, which minimizes the number of sequential addition stages.

The HVWTM was implemented using Verilog Hardware Description Language and synthesized on a FPGA platform. The functionality of the design was verified through simulation, and waveform analysis confirmed correct operation for various input combinations.

Performance evaluation was carried out in terms of area utilization, timing, and power consumption. The results demonstrate that the proposed HVWTM utilizes minimal hardware resources, with low LUT and Flip-Flop usage, indicating high area efficiency. Timing analysis shows positive slack values, confirming that all timing constraints are satisfied and the design operates with reduced propagation delay. Additionally, power analysis indicates low total on-chip power consumption, making the design suitable for energy-efficient applications.

A comparative study with conventional multipliers such as Array, Vedic, and Wallace Tree multipliers highlights that these architectures often involve trade-offs between speed, area, and power. In contrast, the proposed HVWTM achieves a balanced optimization across all three parameters, making it a more efficient and practical solution for high-performance digital systems.

Overall, the proposed HVWTM demonstrates improved computational efficiency, reduced delay, optimized area, and low power consumption, validating its effectiveness for modern digital VLSI applications.

References

1. Janwadkar S, Dhavse R. "ASIC implementation of ECG denoising FIR filter by using hybrid Vedic–Wallace tree multiplier." *Int J Circ Theor Appl*. 2024;52(4):1621-1646
2. J. Tirthaji, *Vedic Mathematics*, Motilal Banarsidass, 1965.
3. C. S. Wallace, "A Suggestion for a Fast Multiplier," *IEEE Trans. Electronic Computers*, 1964.
4. Parhami B. "Computer Arithmetic: Algorithms and Hardware Designs," *Oxford University Press, Inc.*; 1999
5. Bickerstaff KC, Swartzlander EE, Schulte MJ." Analysis of column compression multipliers." *Proceedings of the 15th IEEE Symposium on Computer Arithmetic, ARITH '01: IEEE Computer Society*; 2001:33.
6. Hall SH. "High-Speed Digital System Design: A Handbook of Interconnect Theory and Design Practices." *Wiley IEEE Press*; 2014
7. Janwadkar S, Dhavse R. "Investigation and analysis of power performance area (PPA) cards of digital multiplier architectures". *J Circ Syst Comput*. 2022;31(13):2250239.
8. Abed S, Mohd B, Al-bayati Z, Alouneh S. "Low power Wallace multiplier design based on wide counters." *Int J Circ Theory Appl*. 2012;40: 1175-1185.
9. Tiwari HD, Gankhuyag G, Kim, CM, Cho, YB. "Multiplier design based on ancient Indian Vedic Mathematics." *In: 2008 International SOC Design Conference, Vol. 02. IEEE*; 2008:II-65–II-68.
10. Pushpangadan R, Sukumaran V, Innocent R, Sasikumar D, Sundar V. "High speed Vedic multiplier for digital signal processors." *IETE J Res*. 2009;55(6):282-286.
11. Asid S, Kong Y. "Low-area Wallace multiplier." *VLSI Design*. 2014;2014:343960.

Appendix A

RTL Code

```
//.....Top module for virtual clock.....

`timescale 1ns / 1ps

module top_mult(
    input clk,
    input rst,
    input [7:0] a,
    input [7:0] b,
    output reg [15:0] p
);
wire [15:0] mult_p;
reg [7:0] a_r;
reg [7:0] b_r;
hybrid_8x8 uut (
    .a(a_r),
    .b(b_r),
    .p(mult_p)
);
always @(posedge clk) begin
    if (rst) begin
        a_r <= 8'b0;
        b_r <= 8'b0;
        p    <= 16'b0;
    end else begin
        a_r <= a;
    end
end
```

```

        b_r <= b;
        p    <= mult_p;
    end
end
endmodule

//.....Design module (8x8 Hybrid VWTM) .....

module hybrid_8x8(
    input  [7:0] a, b,
    output [15:0] p,
    output overflow
);

wire [3:0] AL = a[3:0];
wire [3:0] AH = a[7:4];
wire [3:0] BL = b[3:0];
wire [3:0] BH = b[7:4];

wire [7:0] P0, P1, P2, P3;

hybridvwtmul m0(AL, BL, P0);
hybridvwtmul m1(AH, BL, P1);
hybridvwtmul m2(AL, BH, P2);
hybridvwtmul m3(AH, BH, P3);

wire s1,s2,s3,s4,s5,s6,s7,s8,s10,s11,s12,s13,s14,s15,s16;
wire s9,s17,s18,s19;
wire c1,c2,c3,c4,c5,c6,c7,c8,c10,c11,c12,c13,c14,c15,c16;
wire c9,c17,c18,final_carry;

assign p[3:0]= P0[3:0];
assign p[4]=s1;
assign p[5]=s9;
assign p[6]=s10;
assign p[7]=s11;
assign p[8]=s12;

```

```

assign p[9]=s13;
assign p[10]=s14;
assign p[11]=s15;
assign p[12]=s16;
assign p[13]=s17;
assign p[14]=s18;
assign p[15]=s19;
assign overflow=final_carry;

full_adder f1(P0[4],P1[0],P2[0],s1,c1);
full_adder f2(P0[5],P1[1],P2[1],s2,c2);
full_adder f3(P0[6],P1[2],P2[2],s3,c3);
full_adder f4(P0[7],P1[3],P2[3],s4,c4);
full_adder f5(P3[0],P1[4],P2[4],s5,c5);
full_adder f6(P3[1],P1[5],P2[5],s6,c6);
full_adder f7(P3[2],P1[6],P2[6],s7,c7);
full_adder f8(P3[3],P1[7],P2[7],s8,c8);
half_adder h1(c1,s2,s9,c9);
full_adder f9(c9,c2,s3,s10,c10);
full_adder f10(c10,c3,s4,s11,c11);
full_adder f11(c11,c4,s5,s12,c12);
full_adder f12(c12,c5,s6,s13,c13);
full_adder f13(c13,c6,s7,s14,c14);
full_adder f14(c14,c7,s8,s15,c15);
full_adder f15(c15,c8,P3[4],s16,c16);
half_adder h2(P3[5],c16,s17,c17);
half_adder h3(P3[6],c17,s18,c18);
half_adder h4(P3[7],c18,s19,final_carry);
endmodule

```

//.....Design module(4x4 Hybrid VWTM).....

```

module hybridvwtmul(input [3:0] a,b,output [7:0] p);
wire s7,s8,s9,s10,s11,s12,carry;
wire s1,s2,s3,s4,s5,s6;
wire c1,c2,c3,c4,c5,c6,c7,c8,c9,c10,c11;
assign p[0]=a[0]&b[0];

```

```

assign p[1]=s7;
assign p[2]=s8;
assign p[3]=s9;
assign p[4]=s10;
assign p[5]=s11;
assign p[6]= s12;
assign p[7]=carry;
half_adder h1(a[1]&b[2],a[0]&b[3],s1,c1);
half_adder h2(a[2]&b[2],a[1]&b[3],s2,c2);
half_adder h3(a[1]&b[1],a[0]&b[2],s3,c3);
full_adder f1(s1,a[3]&b[0],a[2]&b[1],s4,c4);
full_adder f2(s2,a[3]&b[1],c1,s5,c5);
full_adder f3(c2,a[3]&b[2],a[2]&b[3],s6,c6);
half_adder h4(a[1]&b[0],a[0]&b[1],s7,c7);
full_adder f4(s3,a[2]&b[0],c7,s8,c8);
full_adder f5(s4,c3,c8,s9,c9);
full_adder f6(s5,c4,c9,s10,c10);
full_adder f7(s6,c5,c10,s11,c11);
full_adder f8(c6,a[3]&b[3],c11,s12,carry);
endmodule

```

```

module half_adder(input a,b,output s,c);

```

```

assign s = a^b;
assign c = a&b;
endmodule

```

```

module full_adder(input a,b,c,output s,cout);

```

```

assign s = a^b^c;
assign cout = a&b|b&c|c&a;
endmodule

```


Appendix B

Testbench code

```
`timescale 1ns / 1ps

module top_mult_tb;

    reg clk;
    reg rst;
    reg [7:0] a;
    reg [7:0] b;
    wire [15:0] p;
    integer i;
    integer j;

    top_mult dut (
        .clk(clk),
        .rst(rst),
        .a(a),
        .b(b),
        .p(p)
    );

    // Clock generation
    initial begin
        clk = 1'b0;
    end

    always #5 clk = ~clk;
```

```

// Test procedure
initial begin
    $dumpfile("top_mult_tb.vcd");
    $dumpvars(0, top_mult_tb);
    rst = 1'b1;
    a = 8'b0;
    b = 8'b0;
    // Reset for 2 cycles
    repeat (2) @(posedge clk);
    rst = 1'b0;
    // Test all combinations
    for (i = 0; i < 256; i = i + 1) begin
        for (j = 0; j < 256; j = j + 1) begin
            @(negedge clk);
            a = i[7:0];
            b = j[7:0];
            // Wait for pipeline latency (1 cycle)
            repeat (2) @(posedge clk);
            #1;
            if (p !== (i * j)) begin
                $display("Mismatch: a=%0d b=%0d p=%0d expected=%0d",
                    i, j, p, i*j);
                $finish;
            end
        end
    end
    $display("PASS: all test vectors matched");
    $finish;
end
endmodule

```